Advanced Artificial Intelligence and Machine Learning

Software, Tooling & Hardware



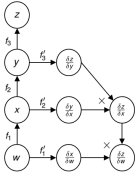
Learning goals

- Deep learning software frameworks and what they abstract away
- How a GPU differs from a CPU: execution units, memory hierarchy, execution and memory model
- Why matmuls are fast and why compute outgrows memory bandwidth
- What makes ML workloads fast on a GPU: the roofline model and the main tricks

Software for Deep Learning

SOFTWARE FOR DEEP LEARNING

- CUDA is a very *low level* programming language and thus writing code for deep learning requires a lot of work.
- Deep learning (software) frameworks:
  - Abstract the hardware (same code for CPU/GPU/TPU)
  - Automatically differentiate all computations
  - Distribute training among several hosts
  - Provide facilities for visualizing and debugging models
  - Can be used from several programming languages
  - Based on the concept of *computational graph*



SOFTWARE FOR DEEP LEARNING

Tensorflow

- Popular in the industry
- Developed by Google and open source community
- Python, R, C++ and Javascript APIs
- Distributed training on GPUs and TPUs
- Tools for visualizing neural nets, running them efficiently on phones and embedded devices.



Keras

- Intuitive, high-level **wrapper** on Tensorflow for rapid prototyping
- Python and (unofficial) R APIs



SOFTWARE FOR DEEP LEARNING

Pytorch

- Popular in academia
- Supported by Facebook
- Python and C++ APIs
- Distributed training on GPUs



MXNet

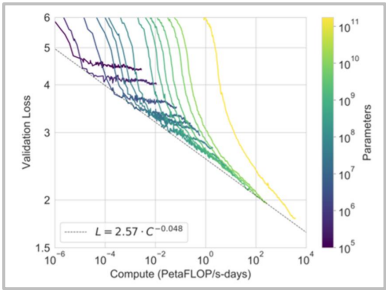
- Open-source deep learning framework written in C++ and cuda (used by Amazon for their Amazon Web Services)
- Scalable, allowing fast model training
- Supports flexible model programming and multiple languages (C++, Python, Julia, Matlab, JavaScript, Go, **R**, Scala, Perl)



Hardware for Deep Learning: GPUs

SETTING THE STAGE: COMPUTE LEADS TO PREDICTABLE PERF

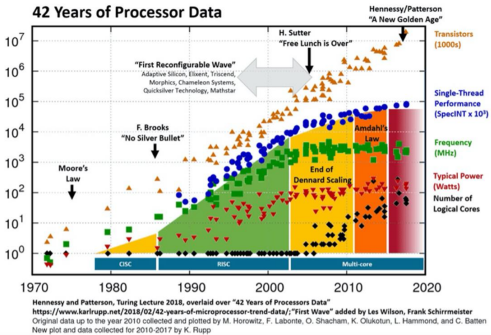
Often times, compute leads to predictable performance gains for language models



Kaplan et al, Neural Scaling Laws

Faster hardware, better utilization, improved parallelization alone can drive progress (for now..)

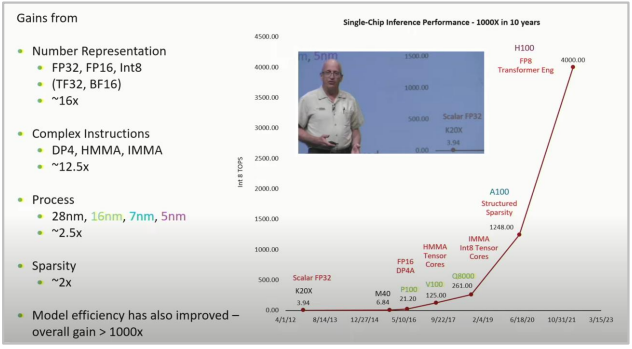
HOW DO WE GET COMPUTE SCALING? EARLY ON – DENNARD SCALING



But the traditional form of scaling (*Dennard scaling*) from 1980-2000s has tapped out.

.. How do we feed LLMs' insatiable appetite for compute?

PARALLEL SCALING CONTINUES



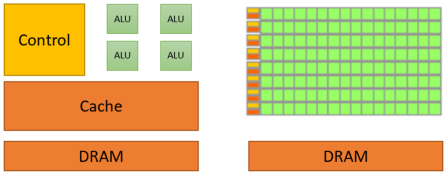
Bill Dally, HotChips keynote

Parallel scaling with GPUs has scaled $>$ 1000x in 10 years.

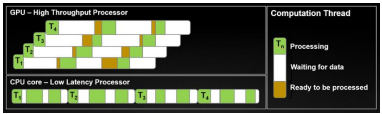
There is no LLM scaling without GPU scaling

HOW IS A GPU DIFFERENT FROM A CPU?

CPUs optimize for a few, fast threads while GPUs optimize for many many threads



Many tiny compute units (ALUs).
Much less support for branching (control, cache)



CPUs optimize for latency (each thread finishes quickly)
GPUs optimize for throughput (total processed data)

https://developer.nvidia.com/blog/cuda-refresher-reviewing-the-origins-of-gpu-computing/

ANATOMY OF A GPU (EXECUTION UNITS)



SM



GA100 Full GPU with 128 SMs

Each SM further contains many **SPs** (streaming processor) that can execute 'threads' in parallel

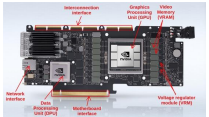
GPUs have *many* **SM** (streaming multiprocessors) that independently execute 'blocks' (jobs).

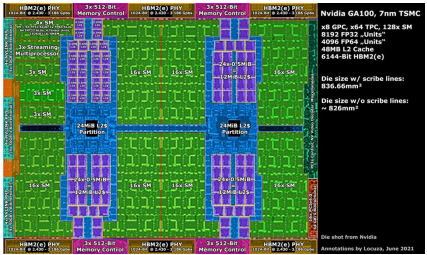
ANATOMY OF A GPU (MEMORY)

The closer the memory to the SM, the faster it is – L1 and shared memory is *inside* the SM. L2 cache is on die, and global memory are the memory chips next to the GPU

TABLE IV
THE MEMORY ACCESSES LATENCIES

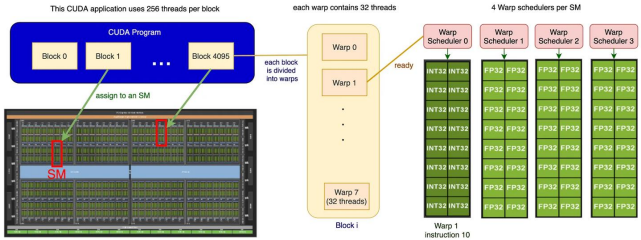
Memory type	*CPI (cycles)*
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)





SRAM (shared/cache memory) is much more expensive (100x) but $\sim$ 8x faster than DRAM (Global memory)

EXECUTION MODEL OF A GPU



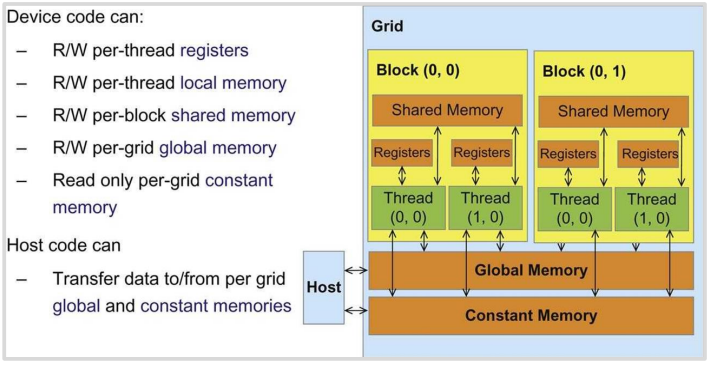
There are 3 important players in the execution model

Threads: Threads 'do the work' in parallel – all threads execute the same instructions but with different inputs (SIMT).

Blocks: Blocks are groups of threads. Each block runs on a SM w/ its own shared memory.

Warp: Threads always execute in a 'warp' of 32 consecutively numbered threads each.

MEMORY MODEL OF A GPU

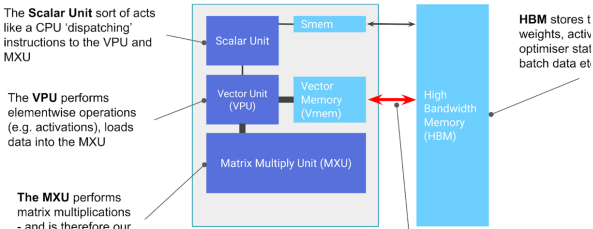


Each thread can access its own register, and shared memory within the block.

Information that goes across blocks need to be read/written to global memory (slow)

SIDE THREAD – WHAT ABOUT TPUS?

GPUs, TPUs, and many other accelerators are at a high level, similar

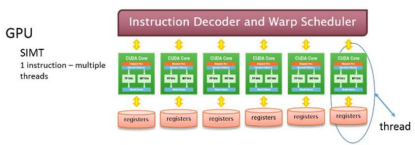


A GPU has more SMs
TPUs has fewer TCs
(but similar matmul perf)

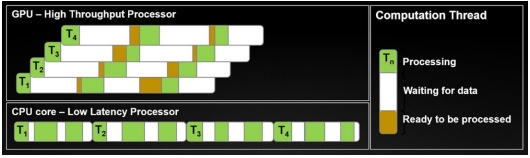
Core structure – lightweight control, fast (big) matmul unit, fast memory.
Differences – how the accelerators are networked (in the parallelism lecture)
– no warps (just blocks – tradeoffs in matmul vs non-matmul)

STRENGTHS OF THE GPU MODEL

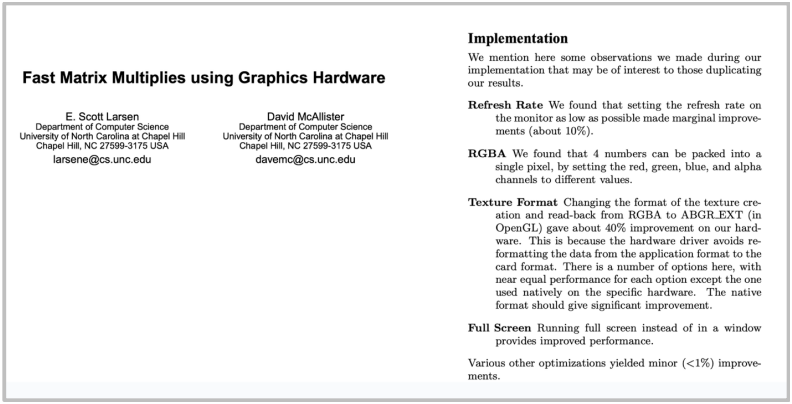
- Easily scales up hard workloads (by adding more SMs)
- Easy (?) to program due to the SIMT model



- Threads are 'lightweight' and can be stopped and started



GPUS AS FAST MATRIX MULTIPLIERS



Fast Matrix Multiplies using Graphics Hardware

E. Scott Larsen
Department of Computer Science
University of North Carolina at Chapel Hill
Chapel Hill, NC 27599-3175 USA
larsene@cs.unc.edu

David McAllister
Department of Computer Science
University of North Carolina at Chapel Hill
Chapel Hill, NC 27599-3175 USA
davemc@cs.unc.edu

Implementation

We mention here some observations we made during our implementation that may be of interest to those duplicating our results.

Refresh Rate We found that setting the refresh rate on the monitor as low as possible made marginal improvements (about 10%).

RGBA We found that 4 numbers can be packed into a single pixel, by setting the red, green, blue, and alpha channels to different values.

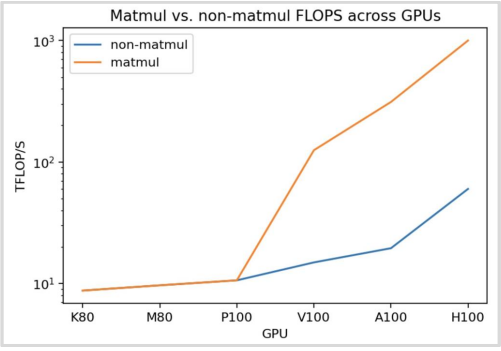
Texture Format Changing the format of the texture creation and read-back from RGBA to ABGR_EXT (in OpenGL) gave about 40% improvement on our hardware. This is because the hardware driver avoids reformatting the data from the application format to the card format. There is a number of options here, with near equal performance for each option except the one used natively on the specific hardware. The native format should give significant improvement.

Full Screen Running full screen instead of in a window provides improved performance.

Various other optimizations yielded minor (<1%) improvements.

Early days of NVIDIA GPUs – programmable shaders. Researchers hacked this to do matmuls

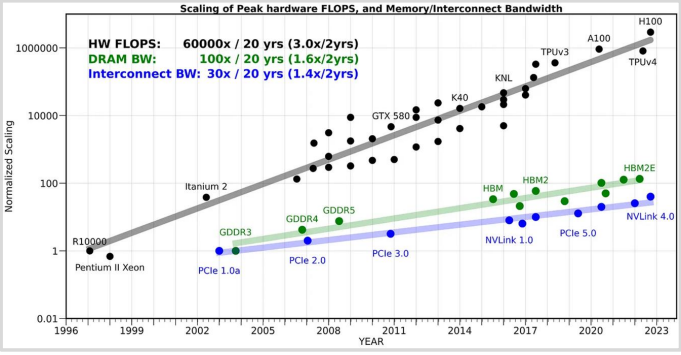
NEW MATMUL HARDWARE MEANS MATMULS ARE FAST AND SPECIAL



Tensor cores (introduced in V, T series) are specialized matrix multiplication circuits.

Matmuls are >10x faster than other floating point ops!

COMPUTE SCALING IS FASTER THAN MEMORY SCALING



https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8

FLOPs scale faster than memory – it's hard to keep our compute units fed with data!

RECAP: GPUS – WHAT ARE THEY AND HOW DO THEY WORK

- GPUs are massively parallel – same instructions applied across many workers
- Compute (and esp matmuls) have scaled faster than memory
- We have to respect the memory hierarchy to make things go fast.



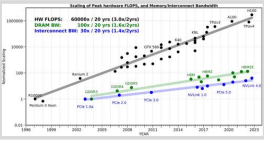
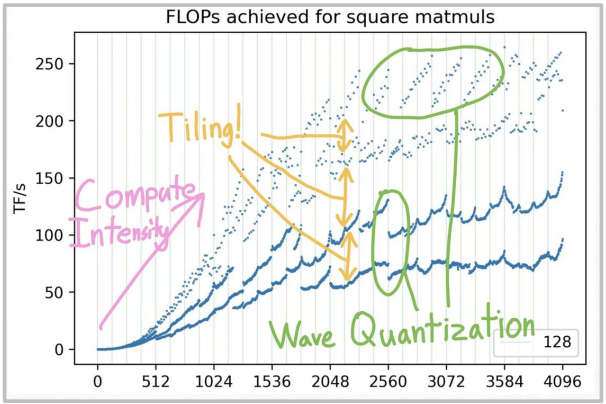


TABLE IV
THE MEMORY ACCESSES LATENCIES

Memory type	*CPI (cycles)*
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)

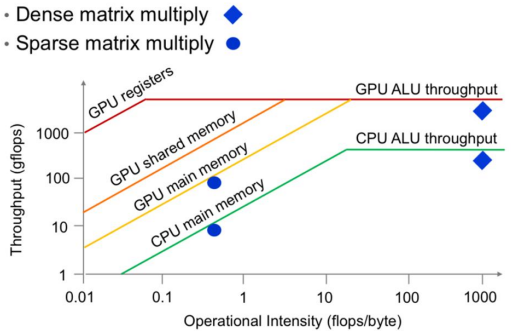
PART 2: MAKING ML WORKLOADS FAST ON A GPU



Performance on a GPU can be complex, even for something as simple as a square matmul

WHAT MAKES ML WORKLOADS FAST?

The roofline model



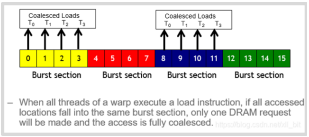
Key to this section: **how do we avoid being memory bound?**

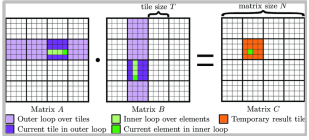
HOW DO WE MAKE GPUS GO FAST?

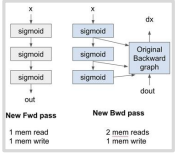
1. Control divergence (not a memory bottleneck..)
2. Low precision computation
3. Operator fusion
4. Recomputation
5. Coalescing memory
6. Tiling

RECAP OF PART 2: MAKING ML WORKLOADS GO FAST

- Reduce memory accesses
  - Coalescing
  - Fusion
- Move memory to shared memory
  - Tiling
- Trade memory for compute/accuracy
  - Quantization
  - Recomputation







LICENSE & ATTRIBUTION

- **Software for Deep Learning** is copied from **Introduction to Deep Learning (I2DL)**, SLDS, LMU Munich, chapter *Hardware and Software* — licensed **CC BY 4.0**: `https://creativecommons.org/licenses/by/4.0/`.
- **Hardware for Deep Learning: GPUs** is adapted from **Stanford CS336 *Language Modeling from Scratch***, Spring 2025, Lecture 5 (*GPUs*) by Tatsunori Hashimoto, `https://github.com/stanford-cs336/spring2025-lectures` — **Copyright 2025 Stanford University**, licensed under the **MIT License** (full notice in `_sources/`). One frame per selected source slide, in the source's order; text retypeset in the source's wording, figures cropped from the source PDF with the credits the source gives.
- Selection and every edit: `order.txt`. **Original to this course:** the learning goals.